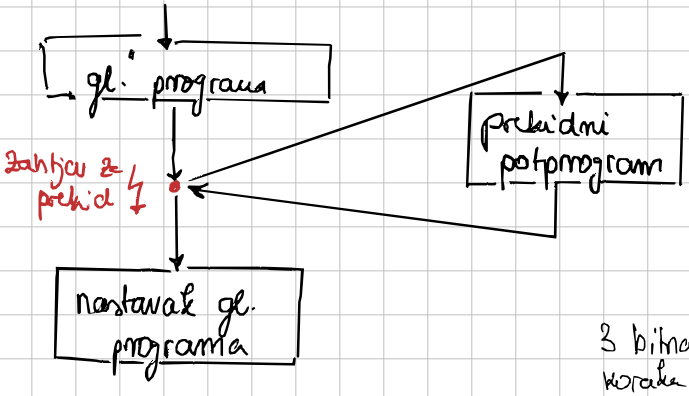
# Prekidni UI - prijenos




* prilikom detekcije prekida, program prvo završi trenutnu naredbu do kraja
* mora biti dozvoljeno prihvaćanje prekida

3 bitna korake prije odradbe prekidnog:
- zabrani daljnje prihv.
- pohrani P.C. i reg. stanja
- odredi adresu prekidnog potprograma
- skoči u prekid. potpr.

# Prekidni potprogram

- spremi kontekst
- očitaj uzrok prekida
- dojavljivanje VJ da je prekid prihvaćen
- posluživanje VJ
- obnavljanje konteksta
- ponovno dozvoljavanje prihvaćanja prekida
- dojavljivanje VJ da je prekid obrađen
- izlazak iz prekidnog potprog. i povratak u glavni

2 vrste iznimke izazvane prekidom → IRQ, → FIQ

↳ prekidi IRQ i FIQ su zabranjeni po defaultu

FIQ: prekid višeg prioriteta

| 31 | 30 | 29 | 28 | | 7 | 6 | 5 | 4 : 0 |
|---|---|---|---|---|---|---|---|---|
| N | Z | C | V | ... | I | F | T | M |

ako je aktivno → 0 u zastavici

| | IRQ | FIQ |
|---|---|---|
| dozvoljeno: | 10010 | 10001 |

IRQ → I=0

FIQ → F=0

# IRQ

- PC i CPSR = automatski pohranjuju u registre

! zabranjuje se prihvaćanje prekida iste razine ! → FIQ ostaje onemogućen

→ procesor prelazi u IRQ

=> prekidni potprogram na adresi 0x18! (tj. na ORG 0x18, bl prekid_IRQ)

- ako se tijekom obrade mijenjaju registri → programski spremiti ih na stog kao kontekst!

↳ ako se u potpr. pozivaju druge fij → spremi R14 (LR)

↳ IRQ stmfd sp!, {r0-r4, LR}
ldr R8, RTC
bl POSL

```
ORG 0
B poslusi_reset
ORG 0x18
B poslusi_IRQ
```

```
poslusi_IRQ
    STMFD sp!, {R0-R5, LR}
    ....
    BL potprogram
    ...
    LDMFD sp!, {R0-R5, LR}
    SUBS PC, LR, #4
```

# FIQ

- PC i CPSR -||-

! zabrana prihvaćanja prekida istog ili slabijeg prioriteta

→ procesor prelazi u FIQ

=> prekidni potprogram na adresi 0x1C

↳ ne mora skakati sa bl; dovoljno direktno napisati => BRZE

- spremanje konteksta → FIQ ima privr. registre R8-R12
  - ne moramo spremati
  - efikasnije

```
ORG 0x1C
POSLUSI_FIQ
    STMFD sp!, {R0, R1, LR}
    ....
    bl potprogram
    ...
    LDMFD sp!, {R0, R1, LR}
    SUBS PC, LR, #4
```

Povratak iz prekidnih potprograma IRQ / FIQ:

**SUBS PC, LR #4**

- povratna adresa iz LR-a se korigira umanjenjem za 4
  => obnova sadržaja registra PC na adresu gdje se gl. proj. treba nastaviti

→ sufiks „S" => obnavlja statusni registar => CPSR = SPSR_mod

# RTC (Real Time Clock)

→ u njemu se zadaje broj impulsa koje treba prebrojiti
↓
MR

→ kada brojilo = MR → RTC sprema
⇒ postavlja zahtjev za prekid

**DR** (Data reg.): može se samo čitati
+0
→ vraća trenutnu vr brojila

**LR** (Load reg.): punjenje / inicijalizacija brojila
+C
→ operacija pisanja upisuje podatak u LR → kopira u brojilo
→ čitanje → vraća upisanu LR vrijednost

**MR** (Match reg.): zadavanje granica brojenja
+4
→ pisanje: upisuje u MR
→ čitanje: vraća zadnju upisanu vr u MR

**STAT/EOI** – 1 bit → 1: čitaj
+8 0: piši
↳ provjera spremnosti i brisanje spremnosti
• čitanje: vraća trenut. spremnost RTC-a
1 = spremnost ; završen ciklus brojenja
• brisanje: briše registar + deaktivira zahtjev za prekid

**CR** + A1 → omogućavanje / onemogućavanje RTC-a da zahtjeva prekide
pisanje { 0 : RTC ne postavlja prekide
1 : RTC postavi prekide kad je spremam

## Ponašanje RTC-a na kraju ciklusa brojanja

→ brojilo i dalje $\boxed{\text{nastavlja}}$ s brojanjem impulsa

→ RTC postaje spremam (STAT / EOI ⇒ 1)

→ ako je zadano da treba postaviti prekid (CR = 1) onda se aktivira INTR

## Inicijalizacija RTC: prije rada s RTC-om treba ga inicijaliz.

▷ Reset brojila

```
mov   r1 , #0
str   r1 , [r0 , #0xc] → LR
```

▷ Zadati konstantu brojanja

```
ldr   R1 , KONST
str   R1 , [r0 #0x4] → MR
```

▷ Zadati način rada:

```
mov   R1, #1
str   R1, [r0, #0x10] → CR   → dozvoljava prekid
```

(16)